

Meerkat: Ranking Alert Families for Capacity-Limited Triage in Multi-Detector Security Operations

Jiacheng Wang

July 2026

Abstract

Security detectors produce far more alerts than analysts can review, so the operational problem is ordering rather than detection. This report describes Meerkat, a triage pipeline for heterogeneous detector output, and reports two successive designs evaluated under leave-one-environment-out testing across the eight simulated companies of the AIT-ADS dataset. The first design ranks individual alerts with supervised models and fails: it attains 86.44% alert level recall at 54.13% workload reduction, yet reaches only 20 of 79 labelled attack windows under a fixed analyst budget, and reaches none of the privilege escalation, reverse shell, webshell, network scanning or credential cracking windows. Diagnosis attributes the failure to the scored unit rather than the model family: 96.1% of in-window alerts belong to a single high-volume phase, so any per-alert ranking objective resolves to finding more of that phase. The second design scores *alert families*, defined as one detector rule firing on one asset within one UTC day. This reduces 2,349,098 alerts to 3,121 reviewable items, removes the volume imbalance without reweighting, and permits a Random Forest that dominates five baselines, including group size, name rarity, a learned per-rule prior, reconciled native severity and an Isolation Forest novelty score, under acceptance criteria fixed before the comparison was run. Privilege escalation and webshell coverage move from 0 of 8 windows to 8 of 8, and 61.2% of the model’s decision weight falls on cross-detector and asset inventory context that the first design never encoded. Both negative results and the metric defects found during the work are reported unchanged.

1 Introduction

A Security Operations Center deploys detectors that convert raw telemetry into alerts. Detection is therefore already complete when an alert exists, and the remaining task is triage: deciding which alerts merit human attention under a fixed review capacity. In the dataset used here a single simulated company produces approximately 6,000 alerts on a day containing no attack, whereas a Tier 1 analyst reviews on the order of tens of items per shift.

Published triage systems largely assume this ordering problem is posed over units that some earlier stage has already assembled. Microsoft’s Adaptive Incident Prioritization scores incidents that Defender has correlated [4]; Sophos’ TEQ predicts actionability at both alert and incident level using analyst escalations as labels [5]; AlertBERT learns to group alerts belonging to one attack [6]; RAPID ranks alerts using surrounding context [7]; DeepCASE analyses sequences of events for the same purpose [10]. Earlier work established analyst feedback and rarity as the core signals [9], and a recent survey maps the space of prioritization criteria and methods [8]. An open-source deployment running Suricata, Wazuh and a log anomaly detector has no correlation layer at all, and must therefore choose its own unit of work.

This report treats that choice as the primary design variable. Its contributions are:

- a cross-detector normalization layer producing a stable machine identity from a customer-supplied asset inventory, separating the machine an alert describes from the sensor that observed it;

- a measured negative result showing that per-alert ranking, whether by a tuned Random Forest, by native severity or by name rarity, cannot reach the quiet phases of an intrusion under an analyst budget;
- an alert-family representation that removes the volume imbalance structurally rather than by reweighting;
- a supervised ranker over families that dominates five baselines under pre-registered acceptance criteria, and an analysis showing that its decision weight rests on cross-detector and asset context;
- an evaluation protocol that reports a budget curve with an explicit saturation threshold, avoiding the arbitrary single budget that invalidated an earlier iteration of this work.

2 Dataset

Experiments use AIT-ADS [1], published by the Austrian Institute of Technology under CC BY 4.0, derived from the AIT log dataset [2]. The dataset contains eight independently simulated enterprise environments, each monitored concurrently by Suricata (network intrusion detection), Wazuh (host intrusion detection) and AMiner (log anomaly detection). Each environment is subjected to a scripted multi-step attack comprising network scanning, service scanning, web enumeration (*dirb* and *wpscan*), webshell upload, credential cracking, reverse shell, privilege escalation, service disruption and DNS exfiltration.

The raw archive contains 2,655,821 alert records. Suricata alerts are collected by Wazuh and appear twice, once decoded from `eve.json` and once re-parsed from `fast.log`; removing the lower-information duplicates leaves 2,349,098 alerts across 79 labelled attack windows. The 306,723 removed records were verified to contain no event-labelled positive.

Two properties motivate this choice of dataset. Three concurrent detectors make cross-detector normalization a genuine rather than artificial problem, and eight independent environments permit training on seven while testing on one the model has never observed.

2.1 Label semantics

Each alert carries two labels produced by different processes. The field `time_label` names the attack phase active when the alert fired, derived from recorded phase intervals. The field `event_label` names the attack event to which the alert could be traced in the underlying logs or network flows. Table 1 shows representative rows from the `russellmitchell` environment.

Table 1: Representative labelled alerts. Both labels are incomplete in different ways.

Time	Alert name	<code>time_label</code>	<code>event_label</code>
00:02:27	Wazuh: ClamAV database update	<code>false_positive</code>	–
03:25:14	Wazuh: ClamAV database update	<code>network_scans</code>	–
03:57:26	Wazuh: Multiple web server 400 errors	<code>dirb</code>	<code>dirb</code>
04:37:40	Wazuh: User successfully changed UID	<code>privilege_escalation</code>	<code>attacker_change_user</code>
04:37:25	AMiner: High entropy in Apache request	<code>reverse_shell</code>	–
00:00:09	AMiner: New ip address in DNS logs	<code>false_positive</code>	<code>dnsteal</code>

The second row is an antivirus update that coincided with attacker scanning, so its phase label conveys temporal proximity without indicating malicious activity. The fifth row occurs inside the reverse shell window yet carries no event label, because the official labelling rules cover only selected log sources. The sixth row carries a `dnsteal` event label while its phase label is `false_positive`, representing genuine exfiltration activity outside the recorded interval.

Two aggregate properties of the labels govern everything that follows. First, event labels are strongly concentrated: **dirb** alone supplies 1,688,945 of 1,817,250 positives (92.9%). Second, and more consequentially, the in-window alert population is concentrated in the same way: four **dirb** windows contain 1,689,359 of the 1,758,559 in-window alerts, or 96.1%. Any objective defined over individual alerts therefore optimises predominantly for one phase. Table 2 gives the joint distribution of the two label sources.

Table 2: Joint distribution of the two label sources across all eight environments.

	Inside attack window	Outside
Event labelled	1,745,038	72,212
Not event labelled	13,521	remainder

The 72,212 event-labelled alerts falling outside every recorded window establish that absence of a phase label cannot be interpreted as benign. An absent event label likewise denotes unlabelled rather than confirmed benign.

3 Method

3.1 Normalization and asset identity

The three detectors emit incompatible JSON schemas. Wazuh nests rule metadata under **rule**, Suricata events arrive wrapped inside a Wazuh envelope at **data.alert**, and AMiner uses an **AnalysisComponent** structure with epoch timestamps. Normalization maps all three onto a common 52-column table, corresponding to the field mapping a SIEM performs when onboarding a log source.

Machine identity requires explicit treatment and was the source of the most damaging defect encountered in this work. Wazuh omits its **predecoder** block for agent-collected files, and an early implementation fell back to **agent.name**, which is the constant string **wazuh-client**. This collapsed 1,698,994 of 1,986,905 Wazuh alerts (85.5%) onto a single fictitious host, so contextual features counted activity across unrelated machines. Three privilege escalation alerts on the **russellmitchell** intranet server reported one, three and four prior alerts on their host in the preceding hour; the true values are 10,902, 10,903 and 10,909.

The pipeline now takes a customer-supplied asset inventory as an input, available before any alert is processed, and distinguishes three identities:

- **entity_id**, the machine the alert describes, always an address the detector itself reported;
- **observer_id**, the sensor or collector that produced the alert;
- **entity_in_inventory**, whether the machine is one the company owns.

Suricata endpoints are resolved against the inventory with a single destination-first rule; AMiner alerts are attributed using the forwarding path **/var/log/logstash/<host>/** when present, which corrects 350 alerts corpus-wide where AMiner reports the collector rather than the affected machine. An earlier discovery pass that inferred the inventory from the alert stream was removed: nine addresses across five environments had their only hostname evidence inside an attack window, so the inferred inventory depended on future information.

The official AIT configuration is used for membership and asset role only, never as a naming authority. Comparing it against the identity Wazuh itself reports shows 80,183 of 287,911 alerts (27.9%) disagreeing, in every case a permutation of external mail server names within one environment. Group membership is unaffected by the permutation, so role-derived features remain valid while display names are taken from the detector.

3.2 From alerts to alert families

The unit scored by the ranker is the primary design variable of this work. The second design scores an *alert family*, defined by the tuple

(UTC day, entity_id, detector_source, rule_id),

that is, one detector rule firing on one asset within one day. This is the unit an analyst actually adjudicates: the decision “this rule fired 380,023 times on this host today” is a single judgement, not 380,023 of them.

The transformation reduces 2,349,098 alerts to 3,121 families, a factor of 753, with a median family size of 3 to 7 alerts and a maximum of 395,054. Crucially, it removes the volume imbalance structurally: each of the four **dirb** floods becomes exactly one row, so the 96.1% concentration reported in Section 2.1 does not survive the change of unit. A family is labelled positive when it contains at least one event-labelled alert, giving 352 positives (11.3%), a rate stable across folds at 10.9% to 11.7%.

3.3 Family features

Each family is described by 30 numeric columns in seven groups: volume and rate (size, duration, alerts per minute); reconciled severity (maximum and mean); transferable rarity (log rule frequency fitted on training environments, plus an unseen-rule indicator); asset role from the inventory (sixteen role indicators and an in-inventory flag); cross-detector corroboration (distinct detectors, families and alerts observed on the same asset that day); detector identity; and native ATT&CK technique presence.

Two design decisions in this list correct defects of the first design. The rarity signal is a continuous count rather than a categorical vocabulary: the first design bucketed any alert name occurring fewer than fifty times into a shared category, and those 38 names, carrying only 744 alerts, touch 33 of the 79 attack windows including all eight privilege escalation windows. The corroboration group exists because the family key separates detectors, and between 57% and 74% of asset-days in this dataset are observed by more than one detector; the corroboration columns restore that evidence without sacrificing the homogeneity a reviewable queue item requires.

Severity reconciliation maps three incompatible native scales onto $[0, 1]$: Wazuh reports levels 0–15 where higher is more severe, Suricata reports 1–3 where lower is more severe, and AMiner reports no severity and is assigned the midpoint. Assigning zero to AMiner, as an earlier version did, ranks an entire detector last and produced a severity baseline weaker than a cruder three-value tier.

3.4 Supervision

The ranker is a Random Forest of 200 trees with balanced class weights and a fixed seed, trained on the family table with the positive target defined above. An alternative target taking the union of event-labelled and in-window families was tested and rejected: it adds 352 label-free families and slightly reduces coverage, indicating those rows contribute noise.

All vocabulary, rarity counts and per-rule statistics are fitted within each training fold. Machine and sensor identifiers are excluded from the feature matrix so that the model cannot memorise an environment.

3.5 Enrichment and queue construction

Families are mapped to ATT&CK techniques and tactics [3] through a reviewed configuration keyed on stable detector rule identifiers, with technique-to-tactic relationships resolved from the official STIX distribution. The mapping is deliberately rule-based so that every claim remains

auditable. Coverage is 93.0% inside attack windows against 1.4% outside. Only the vendor-supplied technique identifiers are used as a model feature; the hand-reviewed mapping was authored with knowledge of this dataset and is reserved for display.

The queue takes the highest-scoring K families per UTC day. The first design additionally required a representative-first policy to prevent one noisy rule consuming the daily capacity; the family representation makes that policy unnecessary, since repetition is already collapsed.

4 Evaluation protocol

Evaluation uses leave-one-environment-out testing: seven environments provide training data while the eighth is held out entirely, repeated eight times.

The primary metric is the number of the 79 labelled attack windows reached by the queue. A window counts as reached only when a queued item holds an alert that carries an official event label *and* falls inside that specific window. Time overlap alone is insufficient and was found to invert results: 89.5% of the ten largest families per day span more than six hours, so an overlap criterion credits them with every window in their day irrespective of content.

Three protocol rules were adopted after an earlier iteration of this work produced conclusions that did not survive scrutiny.

Budget curves, not a single budget. Coverage is reported at $K \in \{5, 10, 25, 50, 100\}$. An earlier acceptance gate fixed at $K = 50$ had no derivation from analyst staffing, and conclusions drawn at it were an artefact of that choice.

An explicit saturation threshold. Because the family table contains approximately 78 items per day, a queue of $K = 50$ reviews 55% to 78% of every family that exists and $K = 100$ reviews 86% to 100%. Coverage measured there approaches reviewing everything and cannot separate rankings. Conclusions are therefore restricted to $K \in \{5, 10, 25\}$, which review 6.4%, 12.9% and 32.2% of the table respectively; saturated budgets are reported as context only.

Dominance, not significance. A method is accepted over another only if it is at least equal at every unsaturated budget and strictly better at one. With eight folds sharing seven training environments each, paired tests are neither independent nor adequately powered, and no claim of statistical significance is made.

Acceptance criteria were fixed in writing before the comparison was executed: the supervised ranker must dominate family size, name rarity and a learned per-rule prior, failing which the learned score is not justified.

5 Results

5.1 First design: per-alert ranking

The first design encoded each alert as 35 numeric columns dominated by activity counts over 60, 600 and 3600 second windows, and trained two Random Forests: a *context model* on attack-window membership and an *event model* on per-alert event labels, fused by maximum.

Read on conventional metrics the context model appears strong: 86.44% alert level recall at 54.13% workload reduction, 98.87% average precision. Read operationally it does not. Table 3 gives windows reached at 50 reviews per day.

Three findings follow. Fusing the two scores by maximum costs six windows relative to the better component, because the dense context signal displaces event-ranked alerts from a capacity-limited queue. Ranking by native severity alone outperforms the complete learned pipeline. And every `network_scans`, `privilege_escalation`, `reverse_shell`, `webshell` and

Table 3: First design. Attack windows reached at 50 reviews per day, summed over eight held-out environments (79 total).

Variant	Windows reached
Event model	26
Native detector severity	21
Full hybrid pipeline	20
Event and context fused	20
Native protected	19
Context model	14

cracking window is reached by no variant at all. The pre-registered threshold of more than 28 windows was not met.

5.2 Diagnosis

Two measurements identify the cause. First, 96.1% of in-window alerts belong to four **dirb** windows, so an objective defined over alerts is overwhelmingly an objective about one phase; the 86.44% recall figure above is substantially a measurement of that phase. Second, approximately 93% of the context model’s importance falls on activity-volume counters, which is the representation the data invites.

A separate defect compounds the first: the categorical encoding bucketed all 38 alert names occurring fewer than fifty times into one shared category. Those names carry 744 alerts in total yet touch 33 of the 79 windows, including all eight privilege escalation windows and five of eight reverse shell windows. The phases the first design never reached are largely the phases whose distinguishing alert names were removed before training.

The conclusion drawn is that the scored unit, not the model family, is the binding constraint.

5.3 Second design: comparison over alert families

Eight ranking arms were compared on the identical family table, folds and budgets, so that any difference is attributable to the score alone. Table 4 reports the result.

Table 4: Attack windows reached (of 79) by alert-family ranking. Budgets to the right of the rule are past the saturation threshold and are context only.

Arm	$K=5$	$K=10$	$K=25$	$K=50$	$K=100$	Mean
Random Forest	34	43	55	59	60	44.0
Random Forest, union target	35	42	53	59	60	43.3
Random Forest + novelty	30	41	56	59	60	42.3
Isolation Forest novelty	24	30	44	51	58	32.7
Reconciled native severity	19	26	48	60	60	31.0
Family size	23	30	39	49	60	30.7
Name rarity	18	27	44	55	60	29.7
Learned per-rule prior	18	27	42	59	60	29.0
Share of table reviewed	6.4%	12.9%	32.2%	62.5%	91.8%	

The supervised ranker satisfies all three pre-registered criteria, dominating family size, name rarity and the learned per-rule prior at every unsaturated budget, and additionally dominating reconciled severity and Isolation Forest novelty. It therefore dominates every non-supervised arm.

Three secondary observations follow. Isolation Forest novelty (32.7) outranks every hand-built heuristic, indicating that multivariate novelty over family features carries more information than univariate name rarity (29.7). Supplying that novelty score to the supervised model as an additional feature does not help (42.3 against 44.0), so the supervised model already extracts what the unsupervised score finds. The learned per-rule prior is the weakest arm, which excludes the possibility that the forest is an expensive lookup table over rule reputation.

Comparison against the first design at equal analyst budget is given in Table 5. The change of unit is worth more than any model choice made at alert level.

Table 5: Windows reached at equal daily review capacity, identical metric.

Budget	Per-alert ML	Per-alert rarity	Alert families
10	21	25	43
25	46	38	55
50	51	52	59

5.4 Coverage by attack phase

Table 6 decomposes coverage by phase and is the central result of this report.

Table 6: Windows reached per attack phase. The final column is the best result achieved by any per-alert variant at 50 reviews per day.

Phase	Windows	$K=5$	$K=10$	$K=25$	Per-alert
wpscan	8	7	8	8	3
dirb	8	7	8	8	5
webshell	8	2	3	8	0
privilege_escalation	8	5	6	8	0
service_scans	8	4	7	7	1
cracking	7	2	3	7	0
service_stop	8	6	6	6	4
reverse_shell	8	1	2	3	0
network_scans	8	0	0	0	0
dnsteal	8	0	0	0	1
Total	79	34	43	55	20

Webshell upload, privilege escalation and credential cracking move from no coverage to complete coverage at 25 reviews per day, a budget that reviews 32% of the family table and issues fewer daily items than the per-alert comparison. Reverse shell improves from zero to three of eight. Network scanning and DNS exfiltration remain unreached, and Section 7 addresses why.

5.5 What the model uses

Table 7 reports Random Forest importance aggregated into the seven feature groups, averaged over the eight folds.

61.2% of decision weight falls on cross-detector corroboration and asset role, the two feature groups produced by the normalization and inventory work and absent from the first design entirely. Volume, which carried approximately 93% of the first design’s importance, now carries 15.4%. Native ATT&CK technique presence contributes essentially nothing at 0.2%, so its value in this pipeline is analyst-facing explanation rather than ranking.

Table 7: Grouped feature importance, mean over eight held-out environments.

Feature group	Importance
Cross-detector corroboration	38.0%
Asset role from inventory	23.2%
Volume and rate	15.4%
Rarity and novelty	10.5%
Reconciled severity	6.6%
Detector identity	6.0%
ATT&CK technique presence	0.2%

6 Discussion

The progression reported here supports a specific claim: for multi-detector telemetry with no correlation layer, the choice of scored unit dominates the choice of model. The same supervised family, a Random Forest, fails at 20 of 79 windows over alerts and succeeds at 55 of 79 over families, on identical data, folds and budgets.

The mechanism is measurable rather than conjectural. Over alerts, one phase supplies 96.1% of the positive population, so a ranking objective resolves to that phase and the resulting model spends 93% of its capacity on volume counters. Grouping collapses each flood to one row, which equalises the phases without any reweighting, and the model’s capacity redistributes onto asset context.

The result also retroactively justifies infrastructure whose value was previously unmeasured. Establishing a stable machine identity across three detectors, and importing an asset inventory, produced no measurable improvement under per-alert ranking. Under family ranking the two feature groups derived from that work carry 61.2% of the model’s decision weight. Infrastructure whose benefit is invisible under one problem formulation may be decisive under another.

Relative to prior work, this pipeline occupies the position that arises when the correlation layer assumed by AIP [4] and by incident-level TEQ [5] is absent. The family key used here is a fixed, explainable substitute for the learned grouping of AlertBERT [6], chosen because it is auditable by an analyst and requires no training. The contribution is not the grouping technique, which is standard, but the measurement that the grouping is worth more than the ranker on this data, and that a widely used per-alert precision metric can be actively misleading.

That last point deserves emphasis. During this work a metric reported as “queue precision” was found to correlate -0.991 with the number of attack windows reached and $+0.87$ to $+0.92$ with duplicate concentration in the queue. It measured redundancy while being read as quality, and it supported the only statistically significant differences observed at the time. It has been withdrawn from all conclusions.

7 Limitations

Results derive from eight simulated environments produced by a single generator with a shared attack script, so they constitute development evidence rather than a claim of real-world performance. All eight environments informed design decisions at some point, so no environment is a fully untouched test set.

Approximately 19 of the 79 windows contain no alert family holding an event-labelled alert inside them, and are therefore unreachable under this supervision at any budget. Every strong arm plateaus at exactly 60 windows when 91.8% of the family table is reviewed. Reported coverage should be read against that ceiling rather than against 79. Network scanning and DNS exfiltration are the principal casualties: `dnsteal` has labelled families in only three of its eight

windows.

Dominance is reported rather than significance. Fifteen paired tests over eight folds yield best and worst two-sided p values of 0.016 and 0.562, and none clears a Bonferroni threshold of 0.0033. One environment, `harrison`, accounts for most of the residual weakness: family size and reconciled severity both reach nine windows there against the forest’s three at $K = 10$.

The unit of review changed between the two designs. A budget of K items per day now denotes K alert families rather than K alerts, and one family may contain hundreds of thousands of alerts. This is defensible because an analyst adjudicates a family as a single decision, but it must accompany any comparison between the two designs.

Alerts outside a labelled window are unlabelled rather than confirmed benign, so outside-window review rates are never reported as false-positive rates. Attack prevalence ranges from 10.2% to 91.2% across environments, far from operational conditions. Timestamps are coarse: Wazuh reports whole seconds 99.9% of the time and 72.2% of all alerts show a zero interval to the previous alert on the same machine, so no feature depending on fine ordering was built. All results use a single seed and 200 trees.

8 Conclusion

Meerkat normalizes heterogeneous detector output against a customer-supplied asset inventory, ranks alert families under a capacity-limited queue, enriches them with auditable ATT&CK context, and is evaluated across environments it has never observed. Ranking individual alerts fails: it reaches 20 of 79 attack windows and none of the intrusion’s quiet phases, and is outperformed by sorting on native severity. Ranking alert families succeeds: a Random Forest over 3,121 families dominates five baselines under criteria fixed in advance, reaches 55 of 79 windows against a ceiling of approximately 60, and moves webshell upload, privilege escalation and credential cracking from no coverage to complete coverage. The decisive change was the unit of work rather than the model, and the model’s own importance profile shows it relies chiefly on cross-detector and asset context. Remaining work is to characterise the unreachable windows, to confirm stability across seeds and forest sizes, and to test whether the family key should separate detectors at all.

References

- [1] M. Landauer, F. Skopik and M. Wurzenberger, *Introducing a New Alert Data Set for Multi-Step Attack Analysis*, Cyber Security Experimentation and Test Workshop (CSET), 2024.
- [2] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger and A. Rauber, *Maintainable Log Datasets for Evaluation of Intrusion Detection Systems*, IEEE Transactions on Dependable and Secure Computing, 2023.
- [3] MITRE, *MITRE ATT&CK Enterprise Matrix and STIX Distribution*, <https://attack.mitre.org>.
- [4] S. Freitas, A. Gharib and M. Magenheimer, *Adaptive Incident Prioritization for Security Operations at Scale*, arXiv:2607.16963, 2026.
- [5] B. Gelman, S. Taoufiq, T. Vörös and K. Berlin, *That Escalated Quickly: An ML Framework for Alert Prioritization*, arXiv:2302.06648, 2023.
- [6] *AlertBERT: A Noise-Robust Alert Grouping Framework for Simultaneous Cyber Attacks*, arXiv:2602.06534, 2026.

- [7] Y. Liu et al., *RAPID: Real-Time Alert Investigation with Context-aware Prioritization for Efficient Threat Discovery*, Annual Computer Security Applications Conference (ACSAC), 2022.
- [8] *Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods*, ACM Computing Surveys, 2024.
- [9] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias and K. Li, *AI²: Training a Big Data Machine to Defend*, IEEE International Conference on Big Data Security, 2016.
- [10] T. van Ede et al., *DeepCASE: Semi-Supervised Contextual Analysis of Security Events*, IEEE Symposium on Security and Privacy, 2022.